

Lecture 03 Conditioning, stability, and cancellation

Prerequisites: Lectures 01–02, derivatives, quadratic equations.

Goal: distinguish a sensitive problem from an inaccurate evaluation procedure.

1. Which error are we measuring?

0–14 minutes: a vocabulary check and a worked backward error

Suppose the mathematical task is $y = f(x)$. The intended input is x , the stored input is $\tilde{x}$, and an algorithm returns $\hat{y}$. The total error can be separated exactly as

$$\hat{y} - f(x) = \underbrace{\hat{y} - f(\tilde{x})}_{\text{evaluation error at stored input}} + \underbrace{f(\tilde{x}) - f(x)}_{\text{effect of changing the input}}.$$

The first term depends on how we compute. The second depends on the mathematical problem and the input change. A better formula may greatly reduce the first term without reducing the second.

Forward error measures the difference between computed and desired outputs: $|\hat{y} - f(x)|$, or its ratio to $|f(x)|$ when the denominator is nonzero. **Backward error** asks for an input change Δx such that $\hat{y} = f(x + \Delta x)$. Its magnitude needs an input scale; if several input changes work, one may seek the smallest under a specified norm. Not every arbitrary output belongs to the range of f , so such a perturbation need not always exist.

Worked example: an approximate square root. Take exact $a = 2$ and the nonnegative candidate $\hat{y} = 1.4142$, interpreted as an exact decimal. Set

$$\Delta a = \hat{y}^2 - a = -0.00003836.$$

Then $\hat{y} = \sqrt{a + \Delta a}$ exactly, so this is an absolute backward error for this answer. Moreover,

$$|\hat{y} - \sqrt{a}| = \frac{|\hat{y}^2 - a|}{\hat{y} + \sqrt{a}}.$$

The denominator explains why a residual can be converted into a forward bound once enough information about the root is known. With both terms greater than 1, the forward error is less than 0.00001918.

Conditioning describes how exact outputs change under input perturbations. **Stability** describes an algorithm's errors; a backward-stable algorithm produces the exact answer to a nearby admissible problem, with a suitably small perturbation over its stated input domain. A small backward error for one example is evidence about that example, not a theorem about every execution.

Explain the difference

Can an algorithm be backward stable and still have a large forward error? Can a well-conditioned problem be solved by a poor algorithm? Give a reason for each answer, then look for concrete examples on the next three pages.

For a concrete example, let the exact input be $x = 1 + 10^{-20}$ and the task $f(x) = x - 1$. Binary64 stores the input as 1 and the subtraction returns 0. The relative backward error is about 10^{-20} , but the relative forward error is one. The computed subtraction is exact on the stored data. The problem is that a tiny admissible input change removes the entire desired output.

2. Derive a condition number and state its limits

14–30 minutes: local sensitivity versus a finite bound

Let f be differentiable at x , with $x \neq 0$ and $f(x) \neq 0$. For a small relative input change η , write $\Delta x = x\eta$. The definition of the derivative gives

$$f(x + x\eta) = f(x) + f'(x)x\eta + o(\eta), \quad \frac{f(x + x\eta) - f(x)}{f(x)} = \frac{xf'(x)}{f(x)}\eta + o(\eta).$$

This motivates the local relative condition number

$$\kappa_f(x) = \left| \frac{xf'(x)}{f(x)} \right|.$$

Roughly, a small relative input error may be amplified by a factor $\kappa_f(x)$. The formula is asymptotic, not a certified finite-perturbation inequality.

Function	Relative condition number	Interpretation
$\sqrt{x}, x > 0$	$1/2$	Relative changes are locally attenuated
$x - 1, x \neq 0, 1$	$ x/(x - 1) $	Sensitive relatively near its zero
$(x - 1)^6, x \neq 0, 1$	$ 6x/(x - 1) $	Strong relative sensitivity near 1

Exact comparison. For $p(x) = (x - 1)^6$, choose $x = 1.001$ and $h = 0.000001$ as exact decimals. The relative input change is $h/x \approx 9.99 \cdot 10^{-7}$. The relative output change is exactly

$$\frac{p(x + h) - p(x)}{p(x)} = (1.001)^6 - 1 \approx 0.006015.$$

The local prediction is $\kappa_p(x)h/x = 6006(h/x) = 0.006$. It explains the scale but is not the exact change. Notice that the output changes by roughly 0.6% even if both evaluations are done without arithmetic error.

A bound that really covers a finite perturbation. If f is continuously differentiable on an interval containing x and $x + h$, and $|f'(t)| \leq M$ throughout it, the mean value theorem proves

$$|f(x + h) - f(x)| \leq M|h|.$$

If also $|f(x)| \geq m > 0$, then the relative change is at most $M|h|/m$. Both bounds must cover the relevant interval, not just its midpoint.

Choose an appropriate scale

At $x = 1$, $p(x) = 0$, so relative output error and the displayed relative condition number are undefined. What absolute statement can you make if the uncertain input satisfies $|x - 1| \leq 10^{-3}$? Does the large nearby relative condition number mean that the absolute output uncertainty must be large?

Absolute conditioning answers a different question. Locally, the absolute output change is controlled by $|f'(x)|$ times the absolute input change. For $f(x) = x - 1$, this factor is exactly one, even near $x = 1$. There is no conflict with its large relative condition number: dividing the output error by the tiny value $|x - 1|$ changes the scale of comparison. Always identify the perturbation model and the error metric before declaring a problem ill-conditioned.

3. Cancellation in an innocent polynomial

30–47 minutes: Tucker's example, exact reference, and interpretation

The polynomial in Tucker's Example 1.6.2 has two equivalent forms:

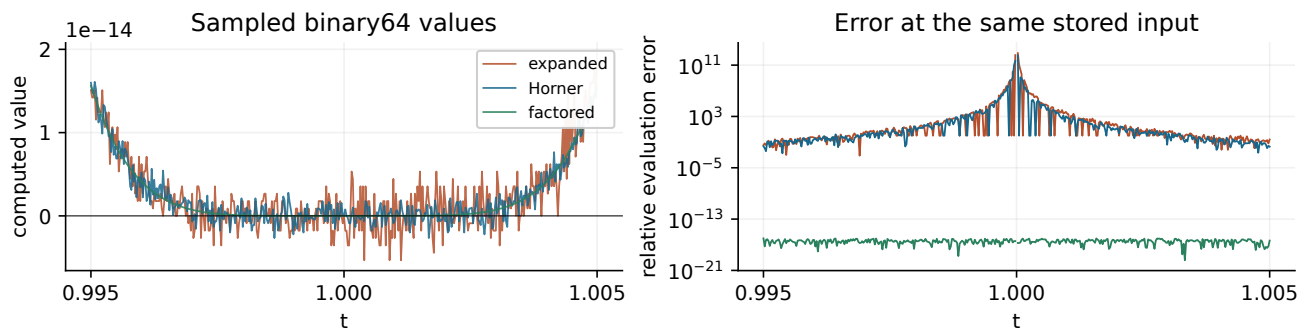
$$p(t) = t^6 - 6t^5 + 15t^4 - 20t^3 + 15t^2 - 6t + 1 = (t - 1)^6.$$

Near $t = 1$, the expanded terms are of order one while their exact sum is tiny. At the exact decimal $t = 1.001$, the answer is 10^{-18} . Errors in much larger intermediate terms can dominate this result. Horner evaluation uses fewer operations, but its intermediate cancellations do not disappear simply because the operation count is smaller.

If operands already contain errors, subtraction exposes them:

$$(a + e_a) - (b + e_b) - (a - b) = e_a - e_b, \quad \frac{|e_a - e_b|}{|a - b|} \leq \frac{|e_a| + |e_b|}{|a - b|}.$$

The subtraction of the stored operands can even be exact. For instance, two slightly different real numbers may first round to the same float and then subtract to exactly zero. The information was lost before the subtraction.



These are sampled floating-point evaluations, not sign or root certificates. The algebraic identity proves $p(t) \geq 0$ for every real t , regardless of any negative values in the computed graph. The logarithmic error plot omits $t = 1$ (relative error is undefined) and samples with exactly zero evaluation error.

To compare algorithms fairly, evaluate the reference at the *same stored input*:

```
from fractions import Fraction
t = 1.001
stored_input = Fraction.from_float(t)
reference = (stored_input - 1)**6
computed = (t - 1)**6
error = abs(Fraction.from_float(computed) - reference)
print(float(error / reference))
```

This measures evaluation error without mixing in the conversion of the intended decimal 1.001. Factoring can improve that evaluation dramatically; it does not change the condition number or eliminate uncertainty in the input.

Explain the apparent contradiction

How can the factored formula be accurate at the stored float when the problem is relatively ill-conditioned near 1? Which term in the page-1 decomposition has improved, and which has not?

4. Change the formula, preserve the problem

47–64 minutes: two algebraic repairs and their limitations

Difference of square roots. For $x \geq 0$, rationalization gives

$$d(x) = \sqrt{x+1} - \sqrt{x} = \frac{1}{\sqrt{x+1} + \sqrt{x}}.$$

The first formula subtracts two close rounded quantities. The second adds them and takes a reciprocal. At binary64 $x = 10^{16}$, even $x + 1$ rounds back to x , so the naive result is zero. The reformulated expression gives about $5 \cdot 10^{-9}$, which has the correct scale. It is still a rounded approximation, not an enclosure. For $x > 0$, differentiation of the reciprocal form gives

$$\kappa_d(x) = \frac{1}{2} \sqrt{\frac{x}{x+1}} < \frac{1}{2}.$$

Thus the loss in the naive formula is not explained by poor relative conditioning of this mathematical function. It is an algorithmic problem.

A small quadratic root. Consider $x^2 + Bx + 1 = 0$ for $B = 10^8$. The usual small-root formula $(-B + \sqrt{B^2 - 4})/2$ subtracts close numbers. Compute the large root by addition of same-sign terms, then use the exact identity $x_1 x_2 = 1$ to obtain the small root:

```
import math
B = 1e8
discriminant = B*B - 4.0
square_root = math.sqrt(discriminant)
naive_small = (-B + square_root) / 2.0
large_root = -0.5 * (B + square_root)
repaired_small = 1.0 / large_root
print(naive_small, repaired_small)
```

In the course environment the values are approximately $-7.45 \cdot 10^{-9}$ and -10^{-8} . The naive formula loses about a quarter of the small root's magnitude. A small absolute error can therefore coexist with a poor relative answer.

For general finite coefficients with $a \neq 0$ and discriminant $D > 0$, the same idea is

$$q = -\frac{1}{2}(b + \text{copysign}(\sqrt{D}, b)), \quad x_1 = q/a, \quad x_2 = c/q.$$

Deal separately with linear equations, zero constant term, and repeated roots. This formula does not by itself prevent overflow in $b^2 - 4ac$, underflow in a small root, or an unreliable discriminant sign near a double root.

Audit before trusting

Which exact identity justifies the second root? What assumptions make division by q valid? How would you check the residual of a stored root without trusting a second floating-point evaluation of the same cancellation-prone expression?

5. Summation order and compensation

64–78 minutes: inspect a short algorithm and its counterexample

Floating-point addition is not associative. An explicit left-to-right sum of $[10^{16}, 1, -10^{16}]$ gives 0, whereas the exact sum of these stored inputs is 1. Losing a small term early can become decisive after later cancellation. Use an explicit loop for this experiment: Python's built-in `sum` need not implement the naive algorithm below.

```
def naive_sum(values):
    total = 0.0
    for value in values:
        total = total + value
    return total

def kahan_sum(values):
    total = 0.0
    correction = 0.0
    for value in values:
        adjusted = value - correction
        updated = total + adjusted
        correction = (updated - total) - adjusted
        total = updated
    return total
```

The compensation variable records low-order information lost in an addition and feeds it into a later step. Follow one update on paper before thinking of it as a black box. If adding `adjusted` leaves `total` unchanged, the computed correction becomes approximately its negative, increasing the next adjusted term.

For the list $[1, 2^{-54}, \dots, 2^{-54}]$ with 10000 small entries, the exact sum is $1 + 10000 \cdot 2^{-54}$. A naive forward loop loses every small term. In this example, reversing the order, Kahan summation, and `math.fsum` all recover the exact representable total. This is a checked example, not a universal exactness claim.

Indeed, the displayed Kahan algorithm still returns zero for $[10^{16}, 1, -10^{16}]$. Compensation helps many sums, but does not certify all sums. For exact stored inputs, a sum of `Fraction.from_float` values supplies an independent rational reference; for general data, rigorous error bounds are needed.

Conditioning of the sum itself. If $s = \sum_i x_i \neq 0$ and each input perturbation satisfies $|\Delta x_i| \leq \eta |x_i|$, then

$$\frac{|\sum_i (x_i + \Delta x_i) - s|}{|s|} \leq \eta \frac{\sum_i |x_i|}{|s|}.$$

This inequality is exact. Heavy cancellation in the true sum can make the factor large even when the summation algorithm is excellent. Stable arithmetic and well-determined input data solve different parts of the problem.

6. Exercises: diagnose, repair, and justify

78–90 minutes: discuss one diagnosis and one repair; remaining work is practice

1. **Conditioning with a finite bound.** For $f(x) = x - 1$ at $x = 1.000001$, compute the relative condition number. Let $|h| \leq 10^{-9}$. Give an exact absolute bound on $|f(x+h) - f(x)|$ and a relative bound using $|f(x)|$. Explain why an accurate subtraction cannot remove this sensitivity.
2. **Separate two errors.** For $t = 1.001$, construct both the exact decimal input and the exact rational value of the stored float. Compute p exactly at both. Then compare the expanded, Horner, and factored floating-point evaluations against the stored-input reference. Use a plain coefficient loop for Horner's method. Which error does each comparison measure? Why is a negative computed value not a counterexample to $p(t) \geq 0$?
3. **A useful reformulation.** Rewrite $\sqrt{1+x} - 1$ without subtracting close values, for $x > -1$. Compare both formulas at $x = 10^{-16}$. Derive the relative condition number for $x \neq 0$ and its limit as $x \rightarrow 0$. Is the inaccurate naive result caused by an ill-conditioned mathematical problem? Label any high-precision reference as an approximation unless you also justify a bound.
4. **Compensation is not certification.** Trace the Kahan code on $[10^{16}, 1, -10^{16}]$, writing the values of **adjusted**, **updated**, and **correction**. Identify where information is lost again. Compare with `math.fsum` and an exact rational sum. What would you need to certify a result when exact rational summation is too expensive?

Optional extension: a smaller step can be worse. For $f(x) = x^2$, the exact forward difference at 1 is

$$D_h = \frac{(1+h)^2 - 1}{h} = 2 + h \quad (h \neq 0).$$

Its truncation error is $|h|$. In machine arithmetic, small numerator errors are divided by h , and sufficiently small h makes $1+h$ round to 1. Explain the competing terms in the heuristic model $C_1 h + C_2 u/h$ for $h > 0$. Its minimum occurs at $h = \sqrt{C_2 u / C_1}$ when both constants are positive; without justified constants, this is guidance rather than a certificate.

Selected checks. Exercise 1 gives $\kappa = 1000001$, absolute bound 10^{-9} and relative bound 10^{-3} . In Exercise 3 the reformulation is $x/(\sqrt{1+x} + 1)$ and $\kappa = (\sqrt{1+x} + 1)/(2\sqrt{1+x}) \rightarrow 1$. For the page-2 question, $0 \leq p(x) \leq 10^{-18}$.

Reading and notebook. Companion: `notebooks/03_stability.ipynb`; Lab 03: equivalent formulas. Tucker (2011), §1.6, especially Example 1.6.2, pp. 20–21. Rump's expression (Example 1.6.1) is an optional notebook extension. For the summation interface, see [Python's math.fsum documentation](#).